

Document de cadrage — Projet Jeu Vidéo Fullstack

Module : Coordination dev Front & Back — M2 Dev Fullstack (Ynov Connect) **Auteur :** Projet individuel **Type de projet :** Jeu vidéo fullstack web **Dépôt :** <https://github.com/Berserkyr/rpg-40k> (public)

1. Présentation du projet

RPG 40K Survivor (nom interne « Survivant de Ruche ») est un jeu de rôle solo textuel au format web, dans un univers grimdark inspiré de la science-fiction militaire. Le joueur incarne **Karimus**, un technicien vox impérial piégé dans une ruche lors d'une invasion. Il doit survivre : explorer, combattre, gérer son inventaire, faire progresser son personnage et interagir avec un **Maître du Jeu (MJ) piloté par une IA**.

Le projet met en œuvre une **communication front/back complète** :

- un **frontend React** (interface terminal grimdark) ;
- un **backend FastAPI** exposant une API REST + streaming SSE ;
- une **base de données SQLite** pour les comptes et les traces de session ;
- une **fonctionnalité IA** (narration dynamique via l'API OpenAI, avec repli local).

2. Objectifs

Objectif	Description
Jeu jouable bout en bout	Créer un compte, se connecter, démarrer une partie, agir, sauvegarder
Communication front/back	REST pour les actions, SSE pour la narration en flux
IA réellement intégrée	Le MJ génère la narration via OpenAI (pas un mock)
Sécurité	Authentification JWT, mots de passe hachés, rôles

Objectif	Description
Persistance	Comptes en base, sauvegardes de partie isolées par joueur

3. Périmètre fonctionnel

Inclus :

- inscription / connexion sécurisées (JWT) ;
- narration IA en streaming ;
- jets de dés 2D6 ;
- système de combat au tour par tour ;
- inventaire et butin procédural ;
- carte de zones et déplacements ;
- quêtes et relations de faction ;
- progression (XP, compétences) ;
- sauvegarde/chargement par utilisateur.

Hors périmètre (assumé) :

- multijoueur temps réel ;
- moteur graphique 2D/3D (le jeu est textuel) ;
- paiement / monétisation.

4. Stack technique

Couche	Technologie	Justification
Frontend	React 18 + Vite	SPA réactive, build rapide, écosystème mûr
Communication	REST + SSE (fetch)	REST pour les actions, SSE pour le flux narratif
Backend	FastAPI (Python 3.13)	Async natif, validation Pydantic, OpenAPI auto
Sécurité	JWT (PyJWT) + bcrypt	Standard d'authentification stateless + hachage sûr

Couche	Technologie	Justification
Base de données	SQLite	Légère, sans serveur, suffisante pour le périmètre
IA	OpenAI Chat Completions	Génération narrative de qualité, streaming
Sauvegards	YAML par utilisateur	Lisible, versionnable, isolé par joueur

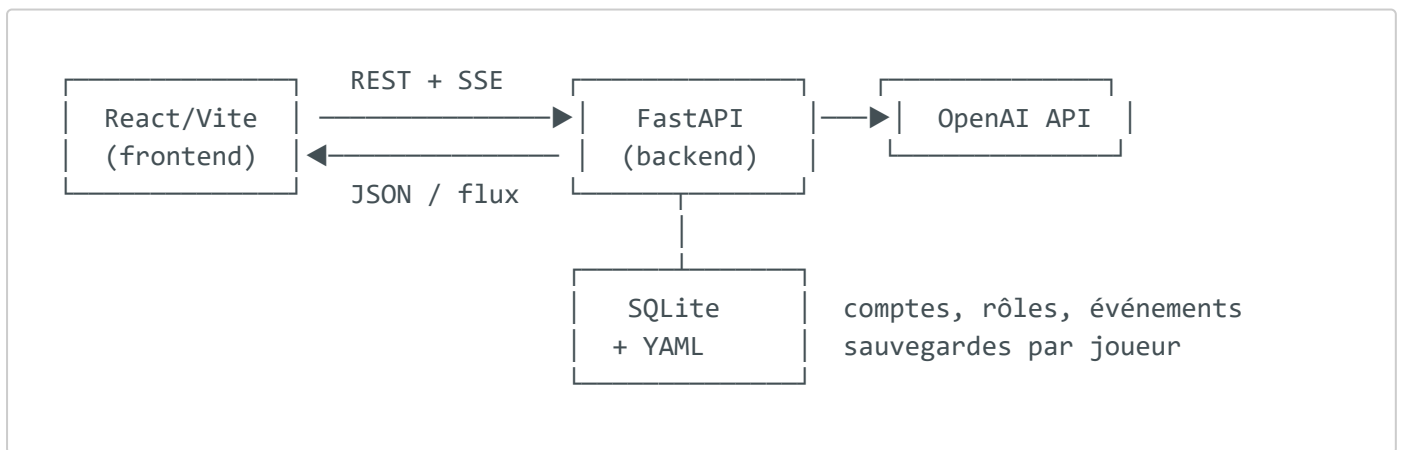
5. Implémentation de l'IA

Le MJ est piloté par l'API **OpenAI** (`gpt-4o-mini`). À chaque action du joueur :

1. le backend construit un prompt système (lore + état du monde) ;
2. il envoie l'historique de conversation à OpenAI ;
3. la réponse est **streamée en direct** vers le frontend via SSE ;
4. l'état du jeu est mis à jour et sauvegardé.

Un **mode de repli local** garantit que le jeu reste jouable même sans clé API (narration procédurale déterministe), ce qui sécurise la démonstration.

6. Architecture globale



7. Sécurité prévue

- Mots de passe **hachés avec bcrypt** (jamais stockés en clair).
- **JWT signés (HS256)** avec expiration, transmis en `Authorization: Bearer`.
- **Rôles** `player` / `admin` avec routes protégées (ex. liste des utilisateurs).

- Isolation des sauvegardes par utilisateur authentifié.
- Secret JWT et clé OpenAI **hors du dépôt** (variables d'environnement `.env`).

8. Livrables (correspondance activités)

Activité	Livrable	Emplacement
1, 4	Doc de cadrage + README + doc technique	ce document, <code>README.md</code> , <code>docs/module/DOC_TECHNIQUE.md</code>
2	MCD / MLD	<code>docs/module/MCD_MLD.md</code>
3, 5	API REST sécurisée JWT	<code>backend/api.py</code> , <code>backend/auth.py</code>
4, 6	Wireframes + frontend fonctionnel	<code>docs/module/WIREFRAMES.md</code> , <code>frontend/</code>
6, 7	Flux clefs + améliorations	<code>frontend/src/api.js</code> , <code>frontend/src/hooks/useSSEChat.js</code> , <code>docs/module/SPRINT_FINALISATION.md</code>
8	Audit et optimisations	<code>docs/module/AUDIT_OPTIMISATION.md</code>
9	Sprint de finalisation + playtest	<code>docs/module/SPRINT_FINALISATION.md</code>
10	Analyse critique	<code>docs/module/ANALYSE_CRITIQUE.md</code>
1-10	Checklist de conformité complète	<code>docs/module/ACTIVITES_1_A_10_CHECKLIST.md</code>
—	Fonctionnalité IA	<code>backend/api.py</code> (<code>_gm_stream</code>)

9. Critères de réussite

- Un utilisateur peut s'inscrire, se connecter, jouer une partie et se déconnecter.
- Les routes de jeu refusent l'accès sans JWT valide (401).
- La narration IA fonctionne et se voit en streaming.
- Les données de partie sont persistées et isolées par compte.

10. Public cible et personas

Persona	Profil	Besoin principal	Attente clé
Le joueur solo	Amateur de RPG textuels / fiction interactive	Vivre une aventure immersive et rejouable	Narration crédible, choix qui comptent
L'évaluateur / jury	Enseignant du module fullstack	Vérifier les compétences front + back	Code lisible, sécurité, doc, démo fluide
Le recruteur technique	Lead dev / CTO consultant le portfolio	Juger la maîtrise technique	Architecture propre, tests, déploiement

Le persona prioritaire pour le **cadrage produit** est *le joueur solo* ; le persona prioritaire pour le **cadrage académique** est *l'évaluateur*. Le projet est conçu pour satisfaire les deux : expérience jouable réelle **et** preuves de compétences.

11. Contraintes et hypothèses

Type	Élément	Impact sur le projet
Contrainte technique	Projet solo, temps limité (module)	Périmètre resserré au MVP jouable
Contrainte technique	Jeu textuel (pas de moteur graphique)	Effort concentré sur logique + IA + UX terminal
Contrainte externe	Dépendance à l'API OpenAI (coût, disponibilité)	Repli local obligatoire pour garantir la démo
Contrainte sécurité	Secrets hors dépôt	<code>.env</code> non commité, variables d'environnement

Type	Élément	Impact sur le projet
Hypothèse	Un seul joueur par compte, pas de concurrence forte	SQLite suffit, pas besoin de PostgreSQL
Hypothèse	Volumétrie faible (usage démo/pédagogique)	Sauvegardes YAML acceptables pour le périmètre

12. Jalons et planning

Jalon	Contenu	État
J1 — Cadrage	Concept, stack, architecture, MCD/MLD	✓
J2 — Cœur backend	API REST, moteur de jeu (<code>src/</code>), SQLite	✓
J3 — Frontend jouable	SPA React, terminal, panneaux, SSE	✓
J4 — Sécurité	Auth JWT, bcrypt, rôles, routes protégées	✓
J5 — IA	Intégration OpenAI + streaming + repli local	✓
J6 — Qualité	Tests backend/front, build, corrections	✓
J7 — Déploiement	Docker Compose sur VPS, healthcheck	✓
J8 — Finalisation	Audit, analyse critique, doc de rendu	✓

13. Matrice des risques projet

Risque	Probabilité	Gravité	Mitigation
Indisponibilité / facturation OpenAI	Élevée	Moyenne	Mode MJ local déterministe automatique
Fuite de secret (clé API, JWT_SECRET)	Moyenne	Élevée	<code>.env</code> hors dépôt, rotation en cas d'exposition
Régression lors des évolutions	Moyenne	Moyenne	Suite de tests back + front, build de contrôle
VPS inaccessible le jour de la démo	Faible	Élevée	App aussi lançable en local (<code>uvicorn</code> + <code>vite</code>)

Risque	Probabilité	Gravité	Mitigation
Complexité de fonctions clés	Moyenne	Faible	Refactoring priorisé et documenté (analyse critique)

14. Méthode de gestion de projet

Le projet est piloté en **mode agile itératif** :

- backlog et priorisation **MoSCoW** (voir [SPRINT_FINALISATION.md](#)) ;
- suivi visuel via [kanban](#) ;
- versionnement discipliné selon la [stratégie Git](#) (commits atomiques, messages explicites, branche **main** stable) ;
- validation par tests automatisés avant chaque incrément important.